

```
---  
title: "Laboratório Especial - Análise de Trajetos de Aeronaves"  
author: "Jhuan Marcos Nunes Solis Silva - RA: 266349"  
format: html  
editor: visual  
---
```

Pacotes Utilizados

```
library(tidyverse)  
library(lubridate)  
library(leaflet)
```

Função de análise

```
analisa_aeronave <- function(tail_number, arquivo) {  
  # Validação básica dos argumentos.  
  if (length(tail_number) != 1 || is.na(tail_number) || !is.character(tail_number)) {  
    stop("tail_number deve ser um único valor de texto, por exemplo 'N431WN'.")  
  }  
  
  if (length(arquivo) != 1 || is.na(arquivo) || !is.character(arquivo)) {  
    stop("arquivo deve ser um único caminho de arquivo.")  
  }  
  
  if (!file.exists(arquivo)) {  
    stop("O arquivo informado em 'arquivo' não foi encontrado: ", arquivo)  
  }  
  
  # 1. Leitura do arquivo flights em partes (chunks).  
  # Apenas as linhas da aeronave solicitada são mantidas em cada chunk.  
  filtra_aeronave <- function(chunk, pos) {  
    chunk |>  
      filter(TAIL_NUMBER == tail_number)  
  }  
  
  voos_filtrados <- read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(filtra_aeronave),  
    chunk_size = 1000000,  
    show_col_types = FALSE  
  )  
  
  # 2. Localização do arquivo airports.csv.  
  # Primeiro procura airports.csv no mesmo diretório de flights.csv(.zip),  
  # que é a organização sugerida no enunciado. Caso não encontre,
```

```
# tenta o diretório de trabalho atual.
diretorio_dados <- dirname(arquivo)
arquivo_aeroportos <- file.path(diretorio_dados, "airports.csv")

if (!file.exists(arquivo_aeroportos)) {
  stop(
    "O arquivo 'airports.csv' não foi encontrado na mesma pasta de 'arquivo': ",
    normalizePath(arquivo_aeroportos, mustWork = FALSE)
  )
}

# Coordenadas exatas dos aeroportos presentes no arquivo airports.
aeroportos <- read_csv(
  arquivo_aeroportos,
  col_select = c(IATA_CODE, LATITUDE, LONGITUDE),
  show_col_types = FALSE
)

# 3. Tratamento e ordenação cronológica.
# IMPORTANTE: não removemos voos da tabela por causa de valores NA.
# O enunciado pede todos os trajetos realizados pela aeronave.
# As linhas que não puderem ser desenhadas no mapa serão filtradas
# somente no momento de construir o mapa.
tabela_voos <- voos_filtrados |>
  # Voos cancelados não foram realizados e, portanto, não representam
  # um trecho efetivamente voado. Os demais registros da aeronave são mantidos.
  filter(is.na(CANCELLED) | CANCELLED == 0) |>
  mutate(
    # Converte DEPARTURE_TIME (HHMM) para hora e minuto.
    HORARIO_SAIDA = suppressWarnings(as.integer(DEPARTURE_TIME)),
    HORA_SAIDA = HORARIO_SAIDA %/% 100,
    MINUTO_SAIDA = HORARIO_SAIDA %% 100,

    # Cria a data e hora usadas para a ordenação temporal.
    DATA_HORA = make_datetime(
      year = YEAR,
      month = MONTH,
      day = DAY,
      hour = HORA_SAIDA,
      min = MINUTO_SAIDA,
      tz = "UTC"
    ),

    # Velocidade média em milhas por hora.
    # Quando AIR_TIME ou DISTANCE não estiver disponível, o valor fica NA.
    VELOCIDADE_MEDIA_MPH = case_when(
      !is.na(DISTANCE) & !is.na(AIR_TIME) & AIR_TIME > 0 ~
        (DISTANCE / AIR_TIME) * 60,
      TRUE ~ NA_real_
    )
  )
```

```
) |>
arrange(DATA_HORA) |>
mutate(
  # Número que identifica a posição de cada trajeto na sequência temporal.
  ORDEM_TRAJETO = row_number()
)

# 4. Acrescenta as coordenadas exatas dos aeroportos de origem e destino.
# Os left_join preservam todas as linhas da tabela de voos.
tabela_voos <- tabela_voos |>
  left_join(
    aeroportos,
    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
  ) |>
  rename(
    LAT_ORIGEM = LATITUDE,
    LON_ORIGEM = LONGITUDE
  ) |>
  left_join(
    aeroportos,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) |>
  rename(
    LAT_DESTINO = LATITUDE,
    LON_DESTINO = LONGITUDE
  ) |>
  arrange(DATA_HORA)

# 5. Construção do mapa Leaflet.
mapa_trajeto <- leaflet() |>
  addTiles() |>
  addScaleBar(position = "bottomleft")

# O mapa utiliza a mesma tabela retornada pela função.
# Cada voo é desenhado como um segmento origem -> destino,
# seguindo a ordem cronológica ORDEM_TRAJETO.
for (i in seq_len(nrow(tabela_voos))) {
  # Para desenhar um segmento, precisamos de coordenadas e de velocidade.
  # Voos com dados ausentes continuam na tabela, mas não são excluídos dela.
  dados_mapa_disponiveis <- all(!is.na(c(
    tabela_voos$LAT_ORIGEM[i],
    tabela_voos$LON_ORIGEM[i],
    tabela_voos$LAT_DESTINO[i],
    tabela_voos$LON_DESTINO[i]
  ))) && !is.na(tabela_voos$VELOCIDADE_MEDIA_MPH[i])

  if (!dados_mapa_disponiveis) {
    next
  }
}
```

```
# A espessura é proporcional à velocidade média.
# Mantemos limites visuais para evitar linhas excessivamente finas/grossas.
espessura <- tabela_voos$VELOCIDADE_MEDIA_MPH[i] / 80
espessura <- max(2, min(10, espessura))

mapa_trajeto <- mapa_trajeto |>
  addPolylines(
    lat = c(
      tabela_voos$LAT_ORIGEM[i],
      tabela_voos$LAT_DESTINO[i]
    ),
    lng = c(
      tabela_voos$LON_ORIGEM[i],
      tabela_voos$LON_DESTINO[i]
    ),
    color = "#005a9c",
    weight = espessura,
    opacity = 0.7,
    popup = paste0(
      "<b>Sequência:</b> ", tabela_voos$ORDEM_TRAJETO[i],
      "<br><b>Voo:</b> ", tabela_voos$FLIGHT_NUMBER[i],
      "<br><b>Origem:</b> ", tabela_voos$ORIGIN_AIRPORT[i],
      "<br><b>Destino:</b> ", tabela_voos$DESTINATION_AIRPORT[i],
      "<br><b>Partida:</b> ", format(tabela_voos$DATA_HORA[i], "%Y-%m-%d %H:%M"),
      "<br><b>Velocidade média:</b> ",
      round(tabela_voos$VELOCIDADE_MEDIA_MPH[i], 1), " mph"
    )
  )
}

# 6. Marca os aeroportos que aparecem nos trajetos que possuem coordenadas.
aeroportos_visitados <- bind_rows(
  tabela_voos |>
    filter(!is.na(LAT_ORIGEM), !is.na(LON_ORIGEM)) |>
    select(
      IATA_CODE = ORIGIN_AIRPORT,
      LAT = LAT_ORIGEM,
      LON = LON_ORIGEM
    ),
  tabela_voos |>
    filter(!is.na(LAT_DESTINO), !is.na(LON_DESTINO)) |>
    select(
      IATA_CODE = DESTINATION_AIRPORT,
      LAT = LAT_DESTINO,
      LON = LON_DESTINO
    )
) |>
distinct()

if (nrow(aeroportos_visitados) > 0) {
```

```
mapa_trajeto <- mapa_trajeto |>
  addCircleMarkers(
    data = aeroportos_visitados,
    lat = ~LAT,
    lng = ~LON,
    popup = ~IATA_CODE,
    radius = 4,
    color = "#d73027",
    fillOpacity = 1
  ) |>
  fitBounds(
    lng1 = min(aeroportos_visitados$LON),
    lat1 = min(aeroportos_visitados$LAT),
    lng2 = max(aeroportos_visitados$LON),
    lat2 = max(aeroportos_visitados$LAT)
  )
}

# Legenda explicando a estatística utilizada no mapa.
mapa_trajeto <- mapa_trajeto |>
  addControl(
    html = "<b>Espessura da linha:</b> proporcional à velocidade média do voo (mph).",
    position = "topright"
  )

# 7. Retorno exigido pelo enunciado:
# lista nomeada com a tabela e o gráfico.
return(
  list(
    tabela = tabela_voos,
    grafico = mapa_trajeto
  )
)
}
```

Execução

```
# Exemplo de uso seguindo o formato indicado no enunciado.
# Estrutura esperada:
# dados/
#   flights.csv.zip
#   airports.csv

# O Quarto executa este documento a partir da pasta do próprio .qmd.
# Os arquivos de dados ficam na subpasta Dados do projeto.
arquivo_voos <- file.path("Dados", "flights.csv")
aeronave_alvo <- "N431WN"
```

```
# Verificações para detectar rapidamente problemas de localização dos arquivos.
if (!file.exists(arquivo_voos)) {
  stop(
    "Não foi possível encontrar o arquivo de voos em: ",
    normalizePath(arquivo_voos, mustWork = FALSE),
    "\nDiretório de trabalho durante o Render: ", getwd(),
    "\nConteúdo do diretório atual: ", paste(list.files(), collapse = ", ")
  )
}

arquivo_aeroportos <- file.path(dirname(arquivo_voos), "airports.csv")
if (!file.exists(arquivo_aeroportos)) {
  stop(
    "Não foi possível encontrar 'airports.csv' em: ",
    normalizePath(arquivo_aeroportos, mustWork = FALSE)
  )
}

cat("Arquivo flights utilizado:", normalizePath(arquivo_voos), "\n")
```

Arquivo flights utilizado: /home/jhuan/LabEsp_projeto/Dados/flights.csv

```
cat("Arquivo airports utilizado:", normalizePath(arquivo_aeroportos), "\n")
```

Arquivo airports utilizado: /home/jhuan/LabEsp_projeto/Dados/airports.csv

```
resultado <- analisa_aeronave(
  tail_number = aeronave_alvo,
  arquivo = arquivo_voos
)

# Visualiza as primeiras linhas da tabela.
head(resultado$tabela)
```

A tibble: 6 × 41

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<chr>
1	2015	1	1	4	WN	1762	N431WN	DCA
2	2015	1	1	4	WN	3564	N431WN	HOU
3	2015	1	1	4	WN	3564	N431WN	BOS
4	2015	1	1	4	WN	3564	N431WN	BWI
5	2015	1	2	5	WN	1793	N431WN	MCI
6	2015	1	2	5	WN	1793	N431WN	LAX

```
# i 33 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
```

```
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,  
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>, ...
```

```
# Mostra quantos trajetos foram encontrados para a aeronave.  
cat("Total de trajetos encontrados:", nrow(resultado$tabela), "\n")
```

Total de trajetos encontrados: 1926

```
# Visualiza o mapa do trajeto.  
resultado$grafico
```



```
# Data/hora local apenas para registro da execução do trabalho.  
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-15 15:51:43 -03"
```